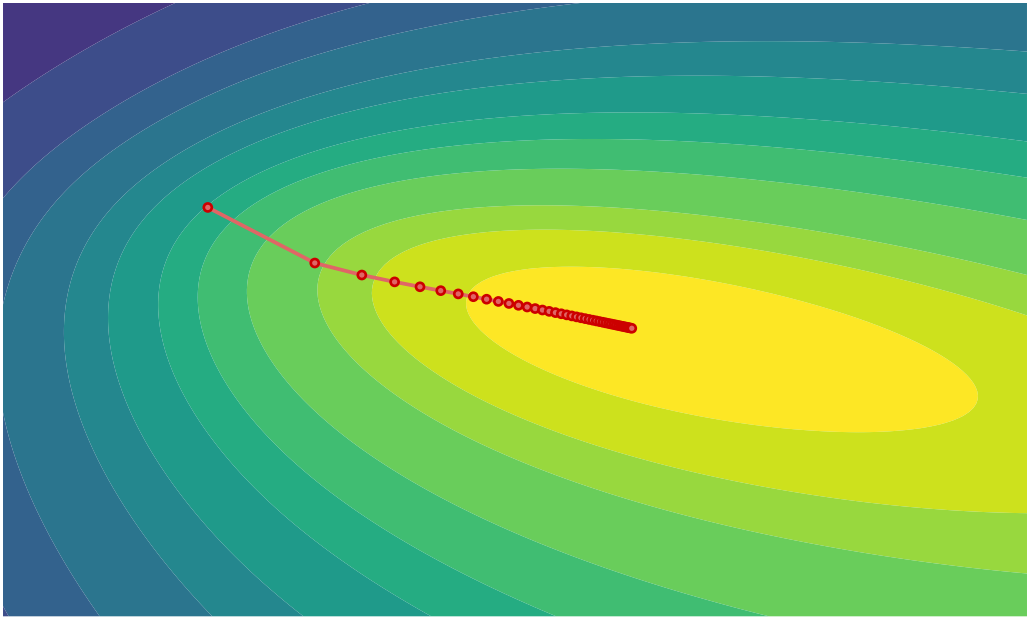


UNIT 01 · CHAPTER 3

Optimization

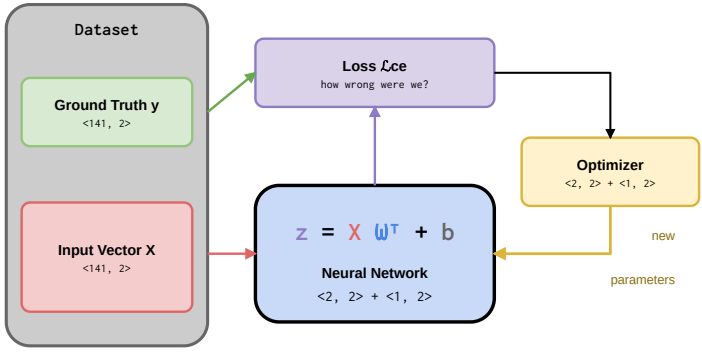
CSE 10124 · Building ChatGPT

William Theisen · University of Notre Dame

Optimizers

Last chapter ended with our football model learning on its own. Started from all zeros (a coin flip on every game, a loss of **0.693**), it worked its way down to **128** of **141** games right, and nobody chose a single weight. We skipped the part that did the work. Something has to look at the loss and decide how to change each parameter, and that something is the **optimizer**.

The loop it sits in is **training**: the model practices over and over on the same games, **changing its parameters a little each time so the loss goes down**. Every other box in the picture below we built last chapter (the games, $z = XW^T + b$, softmax and cross-entropy). This chapter is about the yellow one.



The training loop: games in, a guess out, a loss, new parameters. Then again (and again).

Models as Their Parameters

Our model has a weight per feature and a bias for each class, so `W` is `<2, 2>` and `b` is `<1, 2>`, `6` numbers in all. The shape of the model (a weighted sum, then softmax) is its **architecture**, and it never changes during training. Only the `6` numbers do.

Much like every game is a point in a `2`-dimensional space of yards and turnovers, **every model for this architecture is a single point in a shared `6`-dimensional space**. Here are two of those points, all zeros and a hand-picked model whose "win" class likes yards (`1`) and dislikes turnovers (`−1`):

```
import torch
import torch.nn.functional as F
import pandas as pd
games = pd.read_csv("nd_football.csv")
cols = ["yard_diff", "turnover_diff"]
X = torch.tensor(games[cols].values, dtype=torch.float32)
X[:, 0] = X[:, 0] / 100     # yards, in hundreds
y = torch.tensor(games["win"].values)   # 1 = win
W = torch.zeros(2, 2)       # 2 classes x 2 features
b = torch.zeros(1, 2)
z = X @ W.T + b
loss = F.cross_entropy(z, y)
print("all zeros:", round(loss.item(), 3))
W = torch.tensor([[0., 0.],
                  [1., -1.]])
z = X @ W.T + b
loss = F.cross_entropy(z, y)
print("hand-picked:", round(loss.item(), 3))
```

OUTPUT

```
all zeros: 0.693
hand-picked: 0.341
```

Parameter Space

Same architecture, same games, very different losses. This is also roughly the answer to an old question about brains: if we all share the same architecture, why are people so different? Mostly because the strengths of our connections (our parameters) are different.

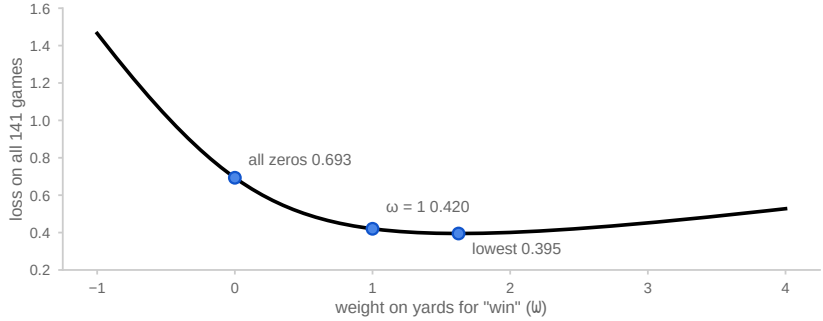
Somewhere in this `6`-dimensional **parameter space** is the best possible model for these games. We just have to find that point. The same goes for every model in this course, the only difference being how many numbers the point has: `7,850` for the one-layer digit classifier in lecture, about `3.4` million for the irishQwen we train on Shakespeare, and about `600` million for Qwen3-0.6B, the full-size model it copies.

So why not just try every model and keep the best one? Even with only `100` values per parameter, our `6` parameters give 10^{12} models to check, and the digit classifier's `7,850` would need `100` to the power `7,850`. **Trying everything is just literally impossible** for any model worth training. We need a way to search that doesn't look at every point, and to get one we first need to see what the loss does as the parameters change.

Loss in Terms of Parameters

If we hold the games constant (the same `141` rows of `X` and `y` every time), the only thing left that can change the loss is the parameters. So we can write the loss as a function of the parameters alone, `L(w, b)`, and ask how each parameter affects it.

Six dimensions are hard to picture, so let's start with one. Below, every parameter is held at `0` except the weight on yards for "win". Each point on the curve is one whole model.

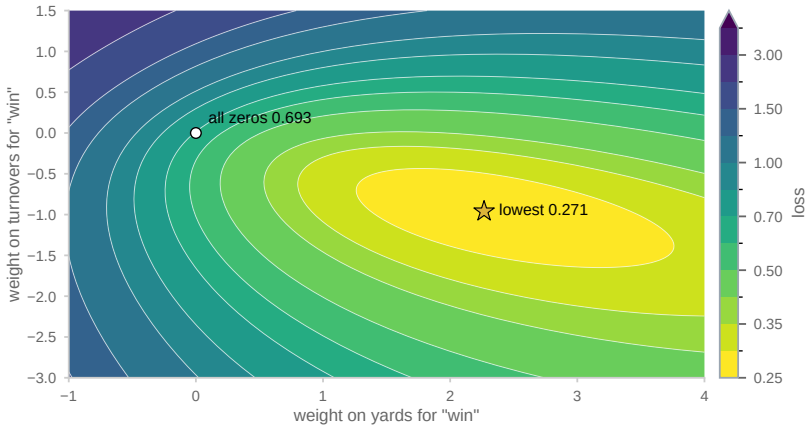


One parameter at a time: the loss on all `141` games as one weight changes.

A negative weight says more yards means more likely to lose, which is wrong for nearly every game, so the loss shoots up. Around `1.6` the curve bottoms out at `0.395`, and past that it climbs again slowly (the model gets so confident about yards that the few games Notre Dame outgained someone and still lost start to hurt). **Training means finding the bottom of this curve**, except in all six dimensions at once.

The Loss Landscape

With two parameters free the curve becomes a surface, the **loss landscape**. Below both of the "win" weights move (yards across, turnovers up and down) and the colour is the loss, yellow being low. The lowest point on this slice is `0.271`, with a weight of about `2.3` on yards and `−0.96` on turnovers.

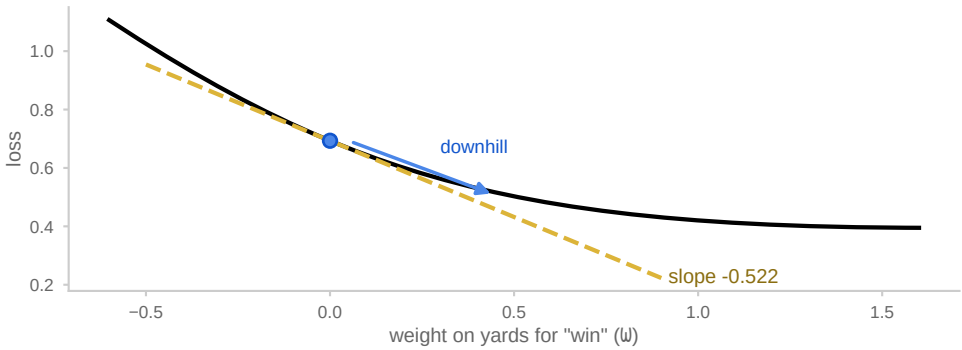


A loss landscape: every point is a model, and its colour is that model's loss on all `141` games.

Drawing this took `22,801` models (`151` values of each weight), which is fine for two parameters and hopeless for six. What we need is a way to **walk downhill from wherever we start, looking only at the ground under our feet**.

Derivatives

Standing at the all-zeros model, which way is downhill? We can find out by **nudging**. Raising the weight on yards for "win" from `0` to `0.01` moves the loss from `0.6931` to `0.6880`, so the loss falls about `0.52` for every unit we raise the weight. That ratio is the slope of the curve where we're standing, its **derivative**.



The slope where we stand: negative, so raising the weight lowers the loss. The exact value is `−0.522` (the nudge was a little too big to get it exactly).

A negative slope means raising the weight takes us downhill, and a positive one means lowering it does. The steeper the slope, the faster the loss changes there.

Gradients

We can do this for each parameter on its own, holding the rest still, which gives one **partial derivative** per parameter, written $\partial L / \partial w$. Stacked into the same shape as the parameters they belong to, they are called the **gradient**. At all zeros ours is:

one entry per weight, rows = [loss, win], columns = [yards, turnovers]

$$\nabla \mathbf{w} = \begin{bmatrix} 0.522 & -0.408 \\ -0.522 & 0.408 \end{bmatrix} \qquad \nabla \mathbf{b} = \begin{bmatrix} 0.252 & -0.252 \end{bmatrix}$$

Each entry's sign says which way is uphill for that parameter and its size says how steep. Together **the gradient points straight uphill** in all six dimensions at once. Here the "win" row wants more weight on yards and less on turnovers, and the "loss" row wants exactly the opposite.

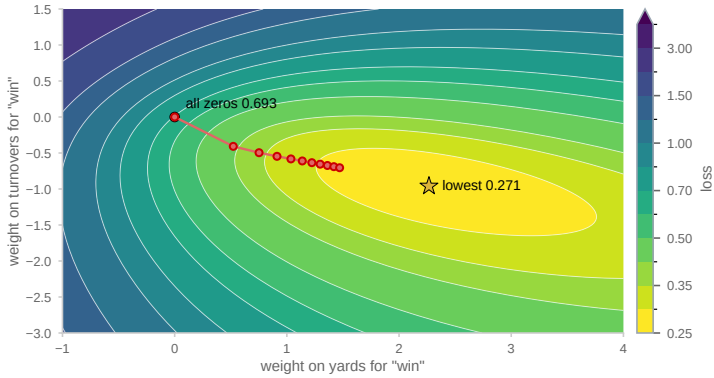
Gradient Descent

The gradient points uphill, so we step the other way. **Gradient descent** updates every parameter by subtracting its gradient, scaled by a small number α called the **learning rate**:

[3.1] every weight, and every bias the same way

$$w \leftarrow w - \alpha \nabla w$$

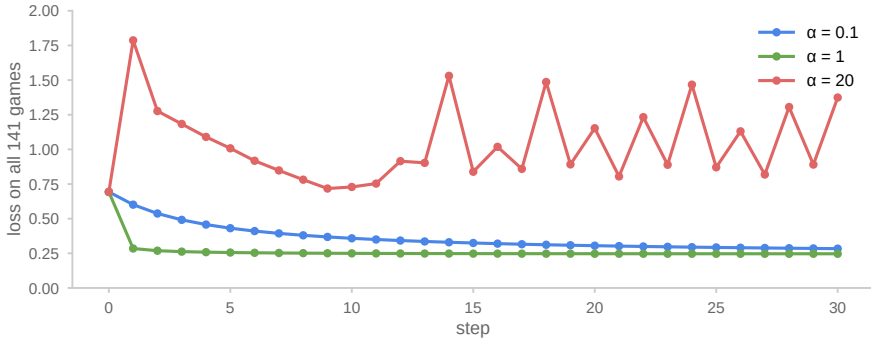
With α = 1, one step from all zeros moves the weight on yards for "win" from `0` to `0.522` (since $0 - 1 \times (-0.522)$), the weight on turnovers to `-0.408`, and the loss on all `141` games drops from `0.693` to `0.285`. Then we compute a new gradient where we landed and step again. **Each step only needs the gradient at the current point**, never the whole landscape.



Ten steps downhill, moving only the two "win" weights so we can draw them (loss `0.693` to `0.288`). The steps shrink as the ground flattens.

The Learning Rate

The learning rate decides how far each step goes, and nothing in the maths picks it for us. Below is the full model (all six parameters) trained from all zeros with three learning rates.



Three learning rates on the same model and the same games.

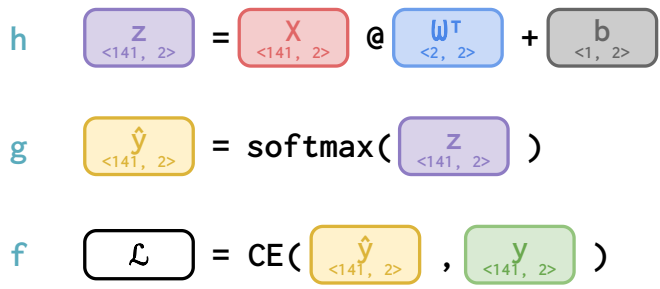
With α = `0.1` we're still creeping downhill after **`30`** steps. With α = 1 we're at the bottom (**`0.247`**) within a handful (last chapter's run used α = `0.1`, hence its **`200`** epochs). With α = `20` the very first step overshoots the bowl entirely, landing on the far wall at a loss of **`1.79`**, and **every step after that bounces from wall to wall instead of settling**.

Unfortunately the right learning rate depends on the model and the data, so in practice people try a few and watch the loss. irishQwen's training script even suggests a different starting value for each of its optimizers, which is about as honest as it gets.

Neural Networks as Composite Functions

Nudging works, but every partial derivative costs one more run of the whole model: `6` extra runs per step for us, `7,850` for the digit classifier and `3.4` million for our small irishQwen. We need the gradient without the nudging.

The way in is to notice that our model is really three functions, one inside the next. h makes the scores from the games and the parameters, g turns the scores into probabilities and f compares the probabilities with what happened:



Written as one expression the loss is `L = f(g(h(X, W, b)), y)`, a **composite function**. Drawn as a series of operations like this it's a **computation graph**. Every neural network, irishQwen included, can be written this way, just with a lot more rows.

The Chain Rule

Calculus has a rule for exactly this situation. The **chain rule** says **the derivative of nested functions is the product of the derivatives of each layer**. Here's a tiny example with a single weight: let $z = 3w$ and $L = z^2$. Then

how L changes with z, times how z changes with ω

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w} = 2z \cdot 3$$

At $w = 1$ we get $z = 3$, $L = 9$ and a derivative of $2 \cdot 3 \cdot 3 = 18$. Checking it the slow way, nudging w to `1.01` gives $z = 3.03$ and $L = 9.1809$, a slope of `18.09` (close enough, the nudge is never exact).

For our model the chain has three links, one per function in the graph:

loss → probabilities → scores → weights

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Each link only involves one small function, and we can work it out once, by hand, ahead of time. That's much cheaper than running the whole model again.

Backpropagation

Working out the links for softmax and cross-entropy is a page of algebra we'll skip (it really isn't that bad), and the result is surprisingly tidy. The first two links together are just **the probabilities minus the one-hot answer**, $\hat{y} - y$, and the last is the input, x, since $z = xw + b$:

[3.2]

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)\, x_i \qquad \nabla w = \frac{1}{N} (\hat{y} - y)^\top X$$

averaged over the N games

Working backwards from the loss through the graph like this, link by link, to every parameter is called **backpropagation**. **It costs about as much as one forward pass**, however many parameters there are.

Backpropagation by Hand

Here it is by hand on three games, for the weight on yards for "win", starting from all zeros (so every $\hat{y}_{win}$ is `0.5`):

game	yards (100s)	won?	$\hat{y} - y$ (win)	× yards
2012 Navy, 50–10	1.49	1	−0.5	−0.745
2012 Alabama (title game), 14–42	−2.27	0	0.5	−1.135
2016 Navy, 27–28	0.02	0	0.5	0.010

The average is `−0.623`: negative, so more weight on yards. Every game pushes in the direction that would have made it less wrong, a win pulling the weight toward its own yards and a loss pushing away from them. Averaged over all `141` games instead of three this is `−0.522`, exactly the gradient we found by nudging.

Backpropagation in PyTorch

Nobody works the links out by hand for a real network. PyTorch records the computation graph as the forward pass runs, and `loss.backward()` walks it backwards, storing each parameter's gradient in its `.grad`. We just have to ask for it with `requires_grad=True`. Continuing from our first cell (which prints its two losses again):

```
W = torch.zeros(2, 2, requires_grad=True)
b = torch.zeros(1, 2, requires_grad=True)
z = X @ W.T + b
loss = F.cross_entropy(z, y)
loss.backward()             # PyTorch does the chain rule
print("PyTorch:", W.grad)
y_hat = torch.softmax(z.detach(), dim=1)
Y = F.one_hot(y, 2)     # win [0, 1], loss [1, 0]
print("by hand:", (y_hat - Y).T @ X / len(X))
```

OUTPUT

```
all zeros: 0.693
hand-picked: 0.341
PyTorch: tensor([[ 0.5224, -0.4078],
        [-0.5224,  0.4078]])
by hand: tensor([[ 0.5224, -0.4078],
        [-0.5224,  0.4078]])
```

The same numbers three ways now: nudging, the formula and PyTorch. From here on we'll let PyTorch do it. irishQwen does it the long way instead: each of its layers has a `backward` method that does one link of the chain, the model calls them in reverse order, and the very first link is `Y_hat - Y`, our $\hat{y} - y$.

Gradient Descent in PyTorch

All together, clearing the gradients after each step (PyTorch adds new ones onto old ones otherwise):

```python
import torch
import torch.nn.functional as F
import pandas as pd
games = pd.read_csv("nd_football.csv")
cols = ["yard_diff", "turnover_diff"]
X = torch.tensor(games[cols].values, dtype=torch.float32)
X[:, 0] = X[:, 0] / 100     # yards, in hundreds
y = torch.tensor(games["win"].values)
W = torch.zeros(2, 2, requires_grad=True)
b = torch.zeros(1, 2, requires_grad=True)
alpha = 1.0                 # the learning rate
for step in range(100):
    loss = F.cross_entropy(X @ W.T + b, y)
    loss.backward()
    with torch.no_grad():   # gradient descent
        W -= alpha * W.grad
        b -= alpha * b.grad
    W.grad.zero_()
    b.grad.zero_()
    if step in (0, 1, 9, 99):
        print(step + 1, round(loss.item(), 3))
pred = (X @ W.T + b).argmax(dim=1)
print("right:", (pred == y).sum().item(), "of", len(y))
```

OUTPUT

```
1 0.693
2 0.285
10 0.25
100 0.247
right: 128 of 141
```

The "win" weights end at `1.05` per `100` yards and `−0.51` per turnover, **what a football fan would have guessed**.

Batches

Every one of those **100** steps used all **141** games. irishQwen learns from the complete works of Shakespeare (over five million characters), and an exact gradient over all of it every step would take forever. Instead we take a **batch** (**16** games here) and **average the loss over just that batch**: a noisier gradient, but a far cheaper step.

```
import torch
import torch.nn.functional as F
import pandas as pd
games = pd.read_csv("nd_football.csv")
cols = ["yard_diff", "turnover_diff"]
X = torch.tensor(games[cols].values, dtype=torch.float32)
X[:, 0] = X[:, 0] / 100
y = torch.tensor(games["win"].values)
W = torch.zeros(2, 2, requires_grad=True)
b = torch.zeros(1, 2, requires_grad=True)
losses = []
for i in range(0, len(X), 16):  # one epoch
    Xb, yb = X[i:i + 16], y[i:i + 16]   # a batch
    loss = F.cross_entropy(Xb @ W.T + b, yb)
    loss.backward()
    with torch.no_grad():
        W -= 1.0 * W.grad
        b -= 1.0 * b.grad
    W.grad.zero_()
    b.grad.zero_()
    losses.append(round(loss.item(), 2))
print("batch losses:", losses)
z = X @ W.T + b
loss = F.cross_entropy(z, y)
print("loss on all games:", round(loss.item(), 3))
```

OUTPUT

```
batch losses: [0.69, 0.29, 0.52, 0.9, 0.31, 0.13, 0.04, 0.17, 0.16]
loss on all games: 0.249
```

Stochastic Gradient Descent

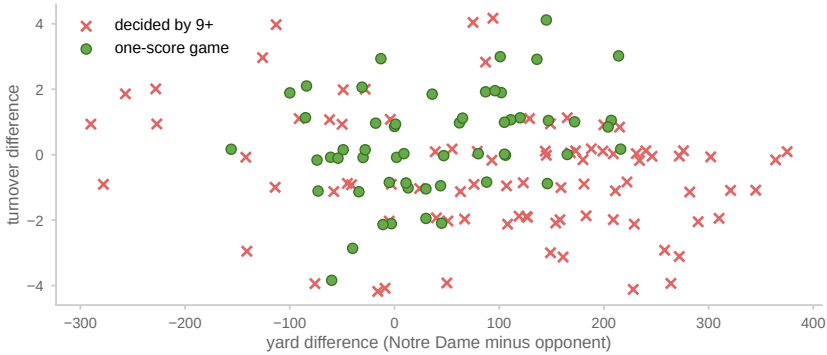
The batch loss jumps around because each batch is a different handful of games (the fourth, games `49` to `64`, holds all `12` games of Notre Dame's `4–8` 2016 season). Still, nine cheap steps got the loss on all `141` games to `0.249`, nearly the `0.247` that a hundred full steps reached. One pass through every batch like this is one **epoch**, the word last chapter used for a pass over every example.

Our batches were the same seasons in the same order every time. Normally we'd shuffle the examples before each epoch, so every batch is a random handful and its gradient is a noisy but fair guess at the real one. Gradient descent on random batches is called **stochastic gradient descent** (SGD).

irishQwen's `optimizers.SGD` optimizer is exactly this update, `W -= lr * dW`, applied to every parameter in the model. Its fancier optimizers (Momentum, Adam) change how big each step is, but they still subtract something from every weight.

Linear and Nonlinear Problems

Let's ask the same model a harder question: was the game **close**, decided by one score (**8** points or fewer)? **55** of our **141** games were.



Close games live in the middle: a big yard gap either way usually means a blowout.

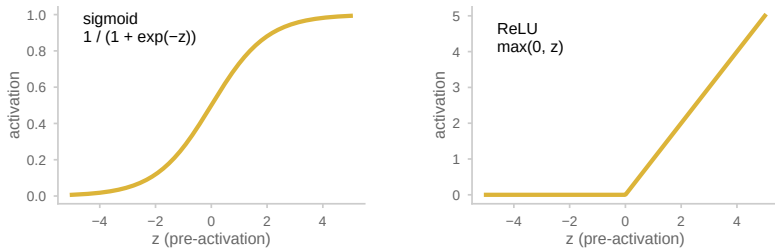
Our model draws one straight boundary through this plot. It can say "more yards, more likely close" or the reverse, but not "close when the yards are even". Trained with gradient descent it gets **87** of **141** right, one game better than always saying "not close".

A problem a straight line (or a flat plane, in more dimensions) can solve is a **linear problem**, and one it can't is a **nonlinear problem**. The famous example is XOR (four points no line can split), which Minsky and Papert used in *Perceptrons* (1969) and which helped bring on the first AI winter. **No amount of optimization fixes a model that can't draw the right shape.**

Activation Functions

The obvious fix is more layers: feed the scores of one linear layer into another. Unfortunately a linear function of a linear function is still linear (the two weight matrices just multiply into one), so stacking them on their own gets us nothing.

What breaks the chain is a non-linear function between the layers, applied to every neuron's weighted sum. This is an **activation function**. Technically we've been using one already, softmax, at the very end.

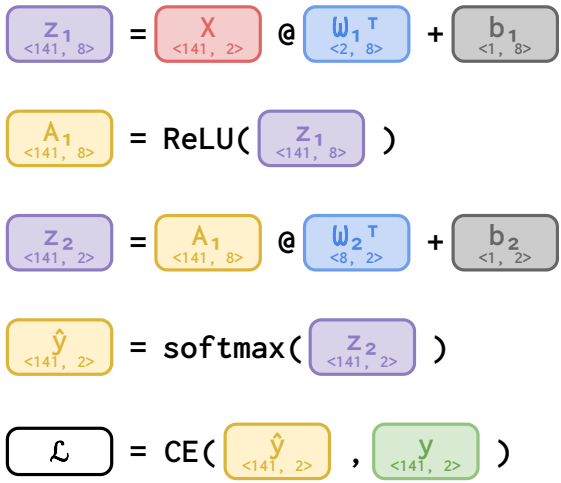


Sigmoid and ReLU: softmax is really a multi-class sigmoid.

The **sigmoid** maps any real number to a value between `0` and `1`. The **ReLU** (Rectified Linear Unit) returns `0` for any value below `0` and the value itself above it. ReLU looks almost too simple to matter, but **a bend at zero is all it takes**: a few ReLUs side by side can make a bump, which is exactly the shape the close games need. irishQwen uses a smoother cousin of it called SiLU.

A Hidden Layer

Our new model puts a layer of **8** ReLU neurons between the inputs and the output. Each neuron has its own weighted sum of yards and turnovers, ReLU bends it, and a second linear layer combines the **8** bent results into the two scores for "not close" and "close". That middle layer is called a **hidden layer**, and the whole thing is a **neural network**.



The close-game network: ReLU after the first layer, softmax after the last.

It has **42** parameters now (**16** weights and **8** biases in the hidden layer, **16** and **2** in the output layer), and the loss is a composite of five functions instead of three. Nothing about the optimizer changes. We still need the gradient of the loss for every parameter, it's just a longer chain.

Backpropagation Through Layers

For the output layer's weights the chain is the same as before, with the hidden layer's activations playing the part of the inputs. For a weight in the first layer the chain has to go back through the second layer and the ReLU before it reaches the input:

one link per row of the graph

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial W_1}$$

↓ each link worked out once

the error, carried back through W_2, kept only where the ReLU was on, times the input

$$\nabla W_1 = \frac{1}{N}\Big(\big[(\hat{y} - y)\, W_2\big] \odot \mathrm{ReLU}'(z_1)\Big)^{\mathsf{T}} X$$

Reading it from the left: the same $\hat{y} - y$ error as before, sent back through the output weights W_2 to find how much each hidden neuron contributed, then multiplied by ReLU's slope (`1` where the neuron was on, `0` where it was off), then by the input. **Each layer passes the error back to the one before it**, which is where the name comes from.

A neuron that was switched off for a game passes no gradient back for it, so that game teaches its weights nothing. With a deep stack of layers, every link in the chain is another number multiplied in, which is where most of the real trouble in training deep networks comes from. Ours has two layers, so we're fine.

Training the Network

This time we'll be honest about it and train only on 2012–2019 (**103** games), keeping 2020–2022 (**38** games, **12** of them close) aside to see how the network does on games it never saw. The starting weights are random (drawn with numpy so they come out the same every run). We'll train for **100** epochs of batches of **16** with $\alpha = 0.5$.

```
import numpy as np
import torch
import torch.nn.functional as F
import pandas as pd
games = pd.read_csv("nd_football.csv")
cols = ["yard_diff", "turnover_diff"]
X = torch.tensor(games[cols].values, dtype=torch.float32)
X[:, 0] = X[:, 0] / 100
close = games["margin"].abs() <= 8   # one score
y = torch.tensor(close.values.astype(int))
n = (games["season"] <= 2019).sum()
rng = np.random.default_rng(0)   # random starting weights
def param(a):
    return torch.tensor(a, dtype=torch.float32,
                        requires_grad=True)
W1 = param(rng.normal(size=(8, 2)))         # <8, 2>
b1 = param(np.zeros((1, 8)))
W2 = param(rng.normal(size=(2, 8)) * 0.5)   # <2, 8>
b2 = param(np.zeros((1, 2)))
params = [W1, b1, W2, b2]
def forward(X):
    A1 = torch.relu(X @ W1.T + b1)    # hidden layer
    return A1 @ W2.T + b2             # z2
```

The Training Loop

The loop itself is the same as before, just with a list of four parameters to step instead of two. Continuing from the cell before:

```
for epoch in range(100):
    for i in range(0, n, 16):          # 2012-2019 games
        Xb, yb = X[i:min(i + 16, n)], y[i:min(i + 16, n)]
        loss = F.cross_entropy(forward(Xb), yb)
        loss.backward()
        with torch.no_grad():
            for p in params:
                p -= 0.5 * p.grad
                p.grad.zero_()
pred = forward(X).argmax(dim=1)
right = (pred == y)
print("2012-2019:", right[:n].sum().item(), "of", n)
print("2020-2022:", right[n:].sum().item(), "of",
      len(y) - n)
```

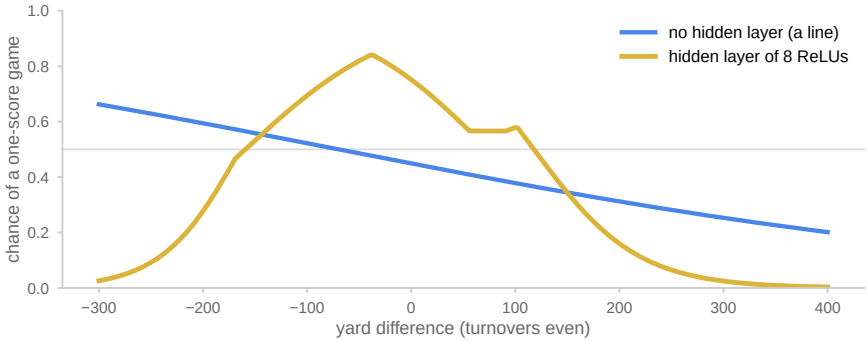
OUTPUT

```
2012-2019: 81 of 103
2020-2022: 24 of 38
```

On the games it trained on, the network gets **81** of **103** right. The linear model managed **87** of all **141**, barely better than guessing "not close" every time.

What the Network Learned

Holding turnovers even and sliding the yard difference across, here is the chance of a close game according to each model:

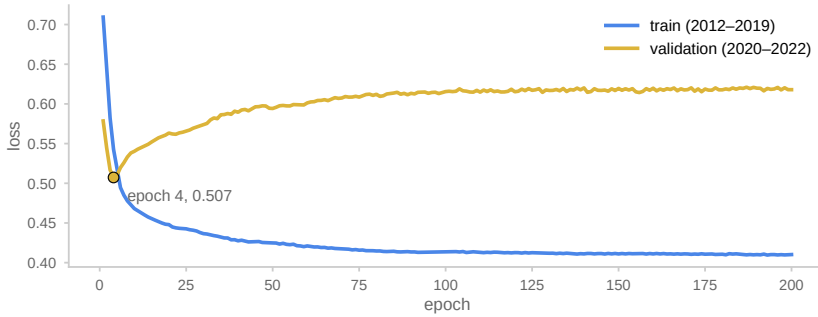


A line and a bump: the linear model can only tilt, the network bends its eight ReLUs into a bump around even yards.

The network found the shape on its own, from nothing but the gradient. Unfortunately it only got **24** of the **38** held-out games right, and just saying "not close" every time gets **26**. **It learned the training games better than it learned football.**

Epochs

Over many epochs we'd expect the loss to steadily decrease, a sign our predictions are getting better. On the training games it does. Tracking the loss on the held-out games after every epoch tells a different story:



Training and validation loss: the held-out games are best after epoch **4**, and every epoch after that makes them worse.

After **4** epochs the network gets **29** of the **38** held-out games right. By epoch **100** it has spent the extra time fitting quirks of the 2012–2019 games, and those don't carry over to 2020. So in practice we watch the **validation loss** and keep the parameters from the epoch where it was lowest.

Everything in this chapter took numbers in: yards, turnovers, pixels. The model we're actually after reads and writes text, and a network can't multiply a word by a weight. Before any of this training can happen, we need a way to turn text into numbers.